10. Technical Proof of Concept

10.1 Purpose of the Proof of Concept

The purpose of this proof of concept is to check whether the main technical parts of the Local Service Problem Diagnosis and Matching System can work together. The system is intended to help users identify the correct service category when they have a household or equipment problem. For example, a user may have a leaking pipe, a faulty electrical switch, or a refrigerator that is not cooling.

Before developing the complete system, I wanted to test whether Python could process a problem description, predict a suitable service category, and save the result in a database. This is important because incorrect classification or problems saving requests could affect the usefulness of the final system.

10.2 Technologies Used

I developed the prototype using Python in Visual Studio Code. Flask was used to create the web application, and HTML and CSS were used to build the input page. The application uses scikit-learn to classify problem descriptions and SQLite to store submitted requests.

The classification component uses Term Frequency-Inverse Document Frequency (TF-IDF) to convert text into numerical features. Logistic Regression then uses those features to predict a service category. The model was trained using a small sample dataset containing three categories: Plumbing, Electrical, and Appliance Repair.

SQLite stores the submitted problem description, predicted category, confidence value, and creation time. The database is created automatically when the application runs.

10.3 Implementation

I created a simple web page that allows a user to enter a description of a household or equipment problem. When the user submits the form, the Flask application receives the text and passes it to the classification component.

The classification component predicts a service category based on the sample training data. After receiving the prediction, the application inserts the submitted description and prediction into the SQLite database. The page then displays the predicted category, model confidence, and saved request ID.

I installed the required Python libraries through the project's requirements.txt file and started the application using python app.py. I opened the application in a web browser through the local address http://127.0.0.1:5000.

This implementation allowed me to test the connection between the web interface, classification model, and database without building every feature of the complete system.

10.4 Test Results

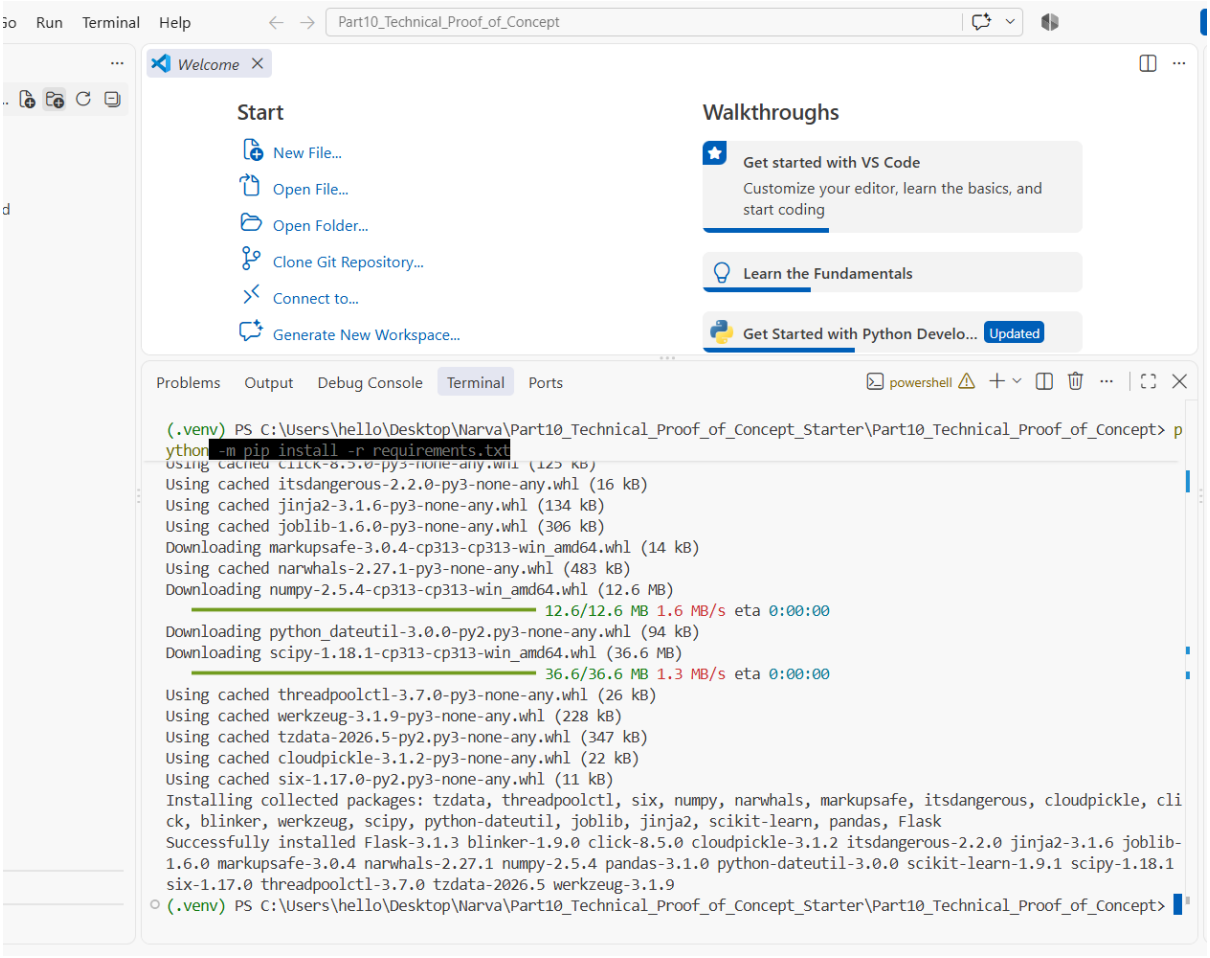
I tested the application by entering the problem description, “The kitchen sink pipe is leaking.”

The application returned the following result:

- Input description: The kitchen sink pipe is leaking.
- Predicted service category: Plumbing.
- Model confidence: 0.51.
- Saved request ID: 1.

The result matched the expected service category for this example. The displayed request ID also indicates that the application inserted a request record into the SQLite database.

The confidence value of 0.51 is a preliminary model output. It does not prove that the classifier is 51% accurate, and it should not be treated as a reliable probability that the prediction is correct.



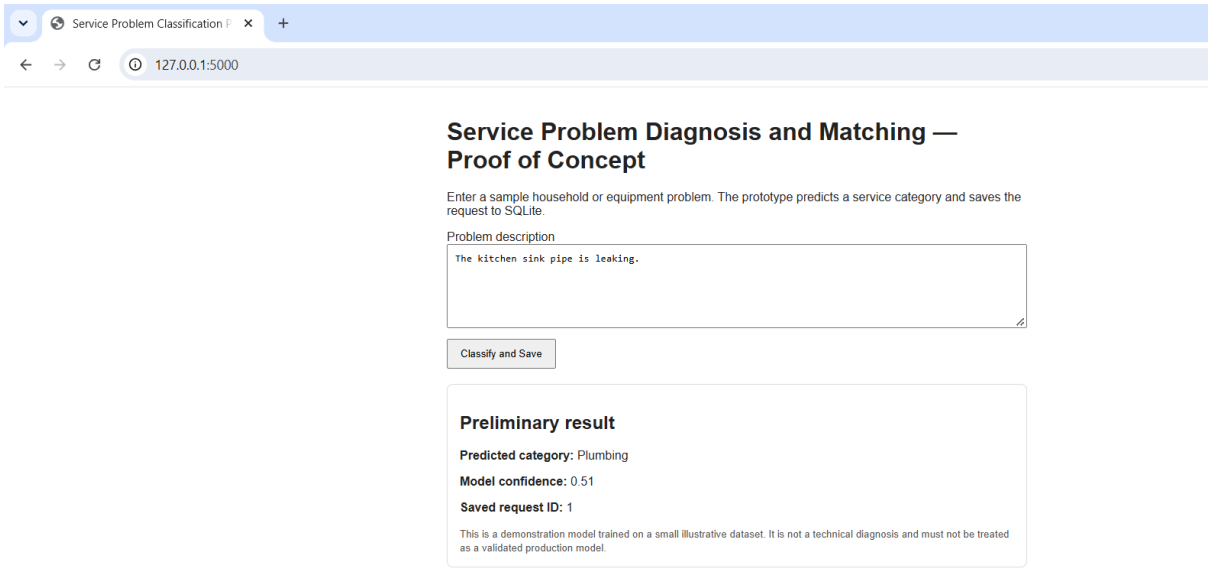


Figure. Problem classification result in the web application.

These screenshots provide evidence that the application ran and produced a classification result. To provide stronger evidence of database persistence, I will also inspect the SQLite table and capture a screenshot of the saved record.

10.5 What the Proof of Concept Demonstrates

The proof of concept demonstrates that it is technically possible to connect a Flask web application with a text-classification model and an SQLite database. The application accepts a problem description, processes the text, returns a predicted category, and saves the result.

This initial implementation addresses two important technical risks. The first is whether a text-classification method can be integrated into the application. The second is whether the application can store the input and prediction for later use.

The successful test provides an initial foundation for the proposed system. It also gives me a starting point for testing additional problem categories and improving the classification process.

10.6 Limitations and Remaining Work

The current prototype is only an initial test. The training dataset contains a small number of example descriptions, so the model has not been shown to work reliably with a wide range of real-world problems. More data is needed to evaluate how well it handles different wording, spelling mistakes, incomplete descriptions, and problems that do not fit the available categories. The next stage will involve collecting or preparing a larger labeled dataset and dividing it into training and testing sets. I will evaluate the classifier using appropriate measures, such as accuracy, precision, recall, and F1-score. I will also test descriptions from

each service category and examine incorrect predictions. The complete system will require additional features, including user authentication, service provider registration, matching requests with available providers, request status updates, and request history. Security, input validation, usability, and performance must also be tested. The API endpoint included in the prototype will need separate testing before I can report it as verified. Therefore, this proof of concept demonstrates that the main technical components can work together in a basic example. It does not yet demonstrate that the complete system is ready for real users or that the classification model has acceptable real-world accuracy.